Fungsi/Method

Naufal Rizqi Habibi
Muhammad Fikri Hidayat

Departemen Matematika
Institut Teknologi Sepuluh Nopember

11 November 2026

Masalah

Masalah

Copy-paste itu mudah, tapi melelahkan kalau yang di-*copas* terlalu banyak. Misalkan ada program 100 baris dan kita ingin menjalankannya 10 kali. Tentu kita tidak mau menulis ulang program itu 10 kali.

Kita butuh cara agar sebuah blok kode bisa **dipakai berkali-kali** tanpa ditulis ulang.

Solusi: Method

Di Java, blok kode yang bisa dipakai ulang disebut **method**. Cukup tulis sekali, panggil sebanyak yang kita mau.

Daftar isi

1. Fungsi
2. Method
3. Rekursif
4. Ringkasan
5. Tugas Praktikum

Definisi

Fungsi adalah pemetaan dari himpunan input ke himpunan output. Di Java, fungsi disebut **method**. Cara kerjanya sama: **menerima input, lalu menghasilkan output**.

Contoh

Misalkan $A = \{\text{Naufal, Fikri, Java}\}$ dan $B = \{1, 2, 3, 4, 5\}$. Fungsi $f : A \to B$ dengan $f :=$ banyak huruf penyusun A:

- Naufal $\mapsto 3$
- Fikri $\mapsto 5$
- Java $\mapsto 4$

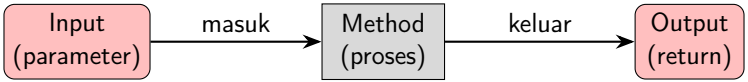


Figure: Method dibayangkan seperti mesin: masukkan bahan, keluar hasil

Analogi

Mesin jus: masukkan *jeruk* (input), mesin bekerja (proses), keluar *jus* (output). Mesin yang sama bisa dipakai untuk jeruk yang berbeda-beda.

Peran Penting

Bayangkan kita membuat program untuk menghitung panjang nama pegawai, dan ada **ribuan** nama yang harus diproses. Tidak efisien kalau kita menulis ribuan blok kode, satu per nama.

Dengan **method**, cukup buat **satu** blok kode penghitung panjang nama, lalu panggil berulang kali dengan nama yang berbeda-beda.

Daftar isi

1. Fungsi
2. Method
3. Rekursif
4. Ringkasan
5. Tugas Praktikum

Definisi

Method adalah blok kode yang dirancang untuk melakukan tugas tertentu dan dapat dipanggil kapan saja dalam program. Method membuat kode lebih terstruktur, bisa dipakai ulang (*reusable*), dan lebih mudah dibaca.

Jenis Method

- **Built-in method**: sudah disediakan Java, contoh `System.out.println()`, `Math.abs()`, `Math.sqrt()`.
- **User-defined method**: dibuat sendiri oleh programmer. Inilah yang kita pelajari hari ini.

Built-in method

Setiap menulis `System.out.println(...)` atau `input.nextInt()`, sebenarnya kita sedang **memanggil method**. Perhatikan: method selalu diikuti tanda kurung `()`.

Listing 1: Contoh pemakaian built-in method

```
double akar = Math.sqrt(16);       // method sqrt: input 16, output 4.0
int mutlak = Math.abs(-7);         // method abs: input -7, output 7
System.out.println(akar);          // method println: mencetak ke layar
```

```
4.0
```

Listing 2: Struktur Method

```
<access_modifier> <return_type> <method_name>(list_of_parameters) {
    // Body of the method
    // Statements to be executed
    return <value>;
}
```

- `access_modifier`: tingkat akses method, contoh `public`, `private`, `protected`.
- `return_type`: tipe data yang dikembalikan. Jika tidak mengembalikan apa pun, gunakan `void`.
- `method_name`: nama method.
- `parameters`: variabel input yang diperlukan method.

Listing 3: Membedah method tambah

```
public static int tambah(int a, int b) {
    return a + b;
}
```

Bagian-bagian

- `public static`: untuk sekarang, tulis saja seperti ini dulu.
- `int` (sebelum nama): method ini **menghasilkan** bilangan bulat.
- `tambah`: nama method (pakai huruf kecil di awal, bermakna).
- `(int a, int b)`: method **menerima** dua bilangan bulat.
- `return a + b;`: nilai yang dikirim balik ke pemanggil.

Catatan

Yang wajib ada: `return_type` dan `method_name`. `parameters` dan `access_modifier` bersifat opsional.

Listing 4: Contoh: access modifier boleh dihilangkan

```
public static void hello() {
    System.out.println("Hello, World!");
}

// bisa diubah menjadi
static void hello() {
    System.out.println("Hello, World!");
}
```

Listing 5: Kerangka program dengan method

```
public class Contoh {

    public static void main(String[] args) {
        // method yang dibuat di bawah DIPANGGIL dari sini
    }

    static void hello() {           // method dibuat DI LUAR main
        System.out.println("Hello, World!");
    }
}
```

Kesalahan umum

Menulis method di **dalam** `main` akan menyebabkan error. Urutan penulisan method di dalam class tidak masalah (di atas atau di bawah `main` sama saja).

Memanggil Method

Method yang sudah dibuat **tidak akan berjalan jika tidak dipanggil**. Cara memanggil: tulis nama method di dalam `main()` (atau method lain), **jangan lupa tanda kurung** `()`.

Membedakan method dan variabel

Method selalu diakhiri tanda kurung `()`, sedangkan variabel tidak. Contoh: `hello()` adalah method, `hello` adalah variabel.

Listing 6: Pemanggilan Method

```java
public class Method {
    public static void main(String[] args) {
        hello();
        hello();
        hello();
    }
    static void hello() {
        System.out.println("Hello, World!");
    }
}
```

```
Hello, World!
Hello, World!
Hello, World!
```

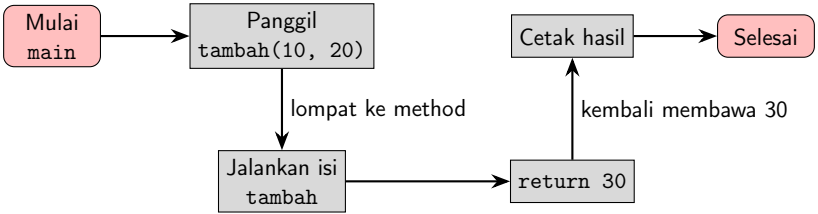


Figure: Program "melompat" ke method, lalu kembali ke tempat pemanggilan

Deskripsi Soal

Buatlah method `static void cetakGaris()` yang mencetak 20 tanda bintang (*) dalam satu baris. Panggil method tersebut tiga kali dari `main`, dengan satu baris tulisan "Halo Alpro" di antara pemanggilan pertama dan kedua.

Hint

- Gunakan `for` di dalam method.
- Gunakan `System.out.print("*")`, lalu `System.out.println()` di akhir.

Contoh Output

```
********************
Halo Alpro
********************
********************
```

Definisi

Parameter adalah variabel yang didefinisikan di dalam tanda kurung saat membuat method. Parameter menerima data dari luar method, sehingga method bisa bekerja berdasarkan nilai yang diberikan. Dengan parameter, method yang sama bisa dipakai untuk data yang berbeda-beda.

Parameter vs Argumen

- **Parameter**: variabel di *definisi* method, contoh `String nama`.
- **Argumen**: nilai yang dikirim saat *pemanggilan*, contoh `"Fikri"`.

Listing 7: Parameter pada Method

```
returnType methodName(parameterType parameterName) {
    // kode di dalam method
}
```

Penjelasan

- `parameterType`: tipe data parameter, seperti `int`, `double`, `String`.
- `parameterName`: nama variabel parameter, dipakai di dalam method.
- Parameter bisa lebih dari satu, dipisahkan dengan tanda koma.
- Saat memanggil, **jumlah, urutan, dan tipe** argumen harus sesuai dengan parameter.

Listing 8: Method dengan satu parameter

```java
public static void sapa(String nama) {
    System.out.println("Halo, " + nama);
}

public static void main(String[] args) {
    sapa("Naufal");       // "Naufal" adalah argumen
    sapa("Fikri");
}
```

```
Halo, Naufal
Halo, Fikri
```

Poin

Satu method sapa, dipakai dua kali dengan data berbeda.

Listing 9: Method dengan dua parameter

```
public static void perkenalan(String nama, int umur) {
    System.out.println("Nama saya " + nama + ", umur " + umur + " tahun");
}

public static void main(String[] args) {
    perkenalan("Fikri", 19);
    perkenalan("Naufal", 19);
}
```

```
Nama saya Fikri, umur 19 tahun
Nama saya Naufal, umur 19 tahun
```

Hati-hati

`perkenalan(19, "Fikri")` akan error karena urutan dan tipe argumen tidak sesuai dengan parameter.

Konsep

Parameter bertindak sebagai **input**, dan `return` bertindak sebagai **output**. Kata kunci `return` mengirim sebuah nilai kembali ke tempat method dipanggil, lalu method **langsung berhenti**.

Kriteria	`void`	**Bertipe data (`int`, dll.)**
Mengembalikan nilai?	Tidak.	Ya, wajib ada `return`.
Cocok untuk	Melakukan aksi, misalnya mencetak ke layar.	Menghitung sesuatu yang hasilnya dipakai lagi.
Contoh	`sapa("Fikri")`	`int z = tambah(1, 2);`

Listing 10: Penjumlahan Dua Bilangan

```
public class Method {
    public static void main(String[] args) {
        int x = 10, y = 20, z = tambah(x, y);
        System.out.println("Hasil " + x + " + " + y + " = " + z);
    }
    public static int tambah(int a, int b) {
        return a + b;
    }
}
```

```
Hasil 10 + 20 = 30
```

Listing 11: Tiga cara memakai nilai yang dikembalikan

```
// 1. Simpan ke variabel
int z = tambah(10, 20);

// 2. Langsung dipakai di dalam println
System.out.println(tambah(10, 20));

// 3. Dipakai di dalam ekspresi lain
int total = tambah(1, 2) * 5;      // (1 + 2) * 5 = 15
```

Catatan

Menulis `tambah(10, 20);` saja (tanpa disimpan) **tidak error**, tetapi hasilnya hilang begitu saja, jadi biasanya tidak berguna.

Listing 12: Method dengan tipe double

```java
public static double luasPersegiPanjang(double p, double l) {
    double luas = p * l;
    return luas;
}

public static void main(String[] args) {
    double hasil = luasPersegiPanjang(4.5, 2);
    System.out.println("Luas = " + hasil);
}
```

```
Luas = 9.0
```

Listing 13: Method yang menjawab ya/tidak

```
public static boolean adalahGenap(int n) {
    if (n % 2 == 0) {
        return true;
    }
    return false;
}

public static void main(String[] args) {
    System.out.println(adalahGenap(10));
    System.out.println(adalahGenap(7));
}
```

```
true
false
```

Listing 14: Variabel hanya hidup di dalam method-nya

```
public static void main(String[] args) {
    int x = 5;
    tampil();
}

static void tampil() {
    System.out.println(x);    // ERROR: x tidak dikenal di sini
}
```

Aturan

Variabel yang dibuat di dalam sebuah method hanya bisa dipakai **di dalam method itu**. Jika method lain butuh datanya, kirim lewat **parameter**.

Listing 15: Kesalahan umum pada method

```
// 1. Lupa return pada method bertipe int
static int f(int a) {
    int b = a * 2;
}                                   // error: missing return statement

// 2. Menyimpan hasil method void
int h = sapa("Naufal");             // error: sapa bertipe void

// 3. Kode setelah return tidak pernah dijalankan
static int g() {
    return 1;
    System.out.println("x");    // error: unreachable code
}
```

Latihan (Bilangan Prima)

Buatlah program yang menampilkan semua bilangan prima dari 1 hingga n menggunakan method bertipe `boolean` bernama `adalahPrima()`.

Ketentuan

1. `adalahPrima(int bilangan)` mengembalikan `true` jika prima, `false` jika tidak.
2. `main` meminta input n, lalu mencetak semua bilangan prima dari 1 sampai n.
3. Prima: bilangan > 1 yang hanya punya 2 faktor (1 dan dirinya sendiri).
4. Tidak boleh rekursi; gunakan *looping* di dalam method.

Hint

Bilangan ≤ 1 langsung `return false`. Cek pembagi dari 2 sampai bilangan -1; jika ada yang habis membagi, `return false`. Jika loop selesai, `return true`.

Contoh I/O

```
Masukkan n :  10
Output :  2 3 5 7
```

Daftar isi

1. Fungsi
2. Method
3. Rekursif
4. Ringkasan
5. Tugas Praktikum

Definisi

Rekursif adalah teknik pemrograman di mana sebuah method **memanggil dirinya sendiri**. Masalah besar dipecah menjadi sub-masalah yang lebih kecil dan sederhana, sampai mencapai **kondisi dasar** (*base case*) yang bisa diselesaikan langsung.

Analogi

Seperti boneka matryoshka: membuka boneka besar, di dalamnya ada boneka yang lebih kecil, dan seterusnya sampai boneka terkecil yang tidak bisa dibuka lagi. Boneka terkecil itulah *base case*.

Listing 16: Pola dasar method rekursif

```
return_type namaMethod(parameter) {
    if (base_case) {
        return nilai_langsung;           // 1. BASE CASE: kapan berhenti
    }
    return namaMethod(parameter_lebih_kecil);   // 2. RECURSIVE CASE
}
```

Dua bagian wajib

1. **Base case**: kondisi berhenti, dijawab langsung tanpa memanggil diri sendiri.
2. **Recursive case**: method memanggil dirinya dengan masalah yang **lebih kecil** (mendekati base case).

Infinite Recursive

Rekursif pada dasarnya adalah perulangan yang dilakukan oleh method itu sendiri. Kita harus tahu **kapan dia berhenti**, kalau tidak terjadi *infinite recursive*. Ide ini mirip dengan `while`: tanpa kondisi berhenti, program tidak akan selesai.

Listing 17: Infinite Recursive (tanpa base case)

```
public static void infinite() {
    System.out.println("Infinite Recursive");
    infinite();
}
```

```
Infinite Recursive
Infinite Recursive
...
Exception: java.lang.StackOverflowError
```

Contoh

Program untuk menghitung faktorial. Ingat: $n! = n \times (n-1)!$ dan $0! = 1! = 1$.

Listing 18: Faktorial

```
public class Method {
    public static void main(String[] args) {
        int n = 5;
        System.out.println("Faktorial dari " + n + " adalah " + factorial(n));
    }
    public static int factorial(int n) {
        if (n == 0 || n == 1) return 1;      // base case
        return n * factorial(n - 1);         // recursive case
    }
}
```

```
Faktorial dari 5 adalah 120
```

```
factorial(4)
= 4 * factorial(3)
= 4 * (3 * factorial(2))
= 4 * (3 * (2 * factorial(1)))      <- base case: factorial(1) = 1
= 4 * (3 * (2 * 1))
= 4 * (3 * 2)
= 4 * 6
= 24
```

Cara membaca

Method memanggil dirinya terus sampai base case. Setelah base case menjawab, hasilnya **dikembalikan satu per satu** ke atas hingga jawaban akhir terbentuk.

Listing 19: Menjumlahkan 1 + 2 + ... + n

```java
public static int jumlah(int n) {
    if (n == 0) return 0;            // base case
    return n + jumlah(n - 1);        // recursive case
}

public static void main(String[] args) {
    System.out.println(jumlah(4));  // 4 + 3 + 2 + 1 + 0
}
```

```
10
```

Poin

Setiap pemanggilan membuat *n* lebih kecil satu, sehingga pasti akhirnya mencapai base case `n == 0`.

Listing 20: Method void yang rekursif

```java
public static void cetak(int n) {
    if (n == 0) {
        System.out.println("Selesai");
        return;                    // return pada void: berhenti saja
    }
    System.out.print(n + " ");
    cetak(n - 1);
}
```

```
cetak(3)  ->  3 2 1 Selesai
```

Catatan

Rekursif tidak harus mengembalikan nilai. Pada method `void`, `return;` hanya berarti "hentikan method ini".

Deskripsi Soal

Buatlah method rekursif `static int pangkat(int a, int b)` yang menghitung a^b (dengan $b \geq 0$) **tanpa** memakai `Math.pow` maupun *looping*.

Hint

- Base case: $a^0 = 1$.
- Recursive case: $a^b = a \times a^{b-1}$.

Contoh I/O

```
Masukkan a :  2
Masukkan b :  5

Output :  32
```

Daftar isi

1 Fungsi

2 Method

3 Rekursif

4 Ringkasan

5 Tugas Praktikum

Konsep	Inti	Ingat
Method	Blok kode bernama yang bisa dipanggil berulang kali.	Ditulis di dalam class, di luar `main`; dipanggil dengan `nama()`.
Parameter	Input untuk method.	Jumlah, urutan, dan tipe argumen harus cocok.
Return	Output dari method; method langsung berhenti.	Tipe `return` harus sama dengan *return type*; `void` tidak mengembalikan nilai.
Rekursif	Method memanggil dirinya sendiri.	Wajib ada base case dan masalah yang makin kecil.

Daftar isi

1 Fungsi

2 Method

3 Rekursif

4 Ringkasan

5 Tugas Praktikum

Penugasan

Tugas praktikum minggu ini dapat diakses melalui tautan berikut:

Klik di sini untuk membuka lembar soal tugas